



Department of Data Science

ALGORITHMIC PROBLEM-SOLVING HACKATHON (APSH 2026)

PROJECT DOCUMENTATION

Date: 03-10-2026 | Venue: 1st Floor, A & B Blocks

Field	Details
Project Title	Hospital Patient Priority Queue (Emergency Triage Manager)
Team Name / Team No.	Team Name: _____ / Team No.: 4
Year / Semester / Section	II Year B.Tech / __1 st __ Sem / ____
Algorithm Paradigm Used	Heap / Priority Queue
Programming Language / Tools	Python 3; VS Code
Faculty Mentor	_____

Team Members

S.No	Name of the Student	Roll No.	Role
1	A. Sai Charitha	25MVCSDR0005	Team Lead
2	G. Sathvika	25MVCSDR0020	
3	G. Ganesh	25MVCSDR0018	
4	D. Sai Kumar	25MVCSDR0014	

1. Problem Statement

A hospital emergency department receives patients with different levels of urgency. Patients with higher emergency priority must be treated before patients with lower priority. If two patients have the same priority, the patient who arrived earlier should be treated first. Task: Design a system that efficiently manages patients and determines the next patient to be treated whenever a doctor becomes available.

Explanation

- Patients arrive with an urgency level from 1 (minor) to 5 (critical).
- The most critical patient must always be treated first.
- Patients with equal priority are treated in arrival order (first come, first served).
- Whenever a doctor is free, the system must quickly give the next patient.

2. Objectives

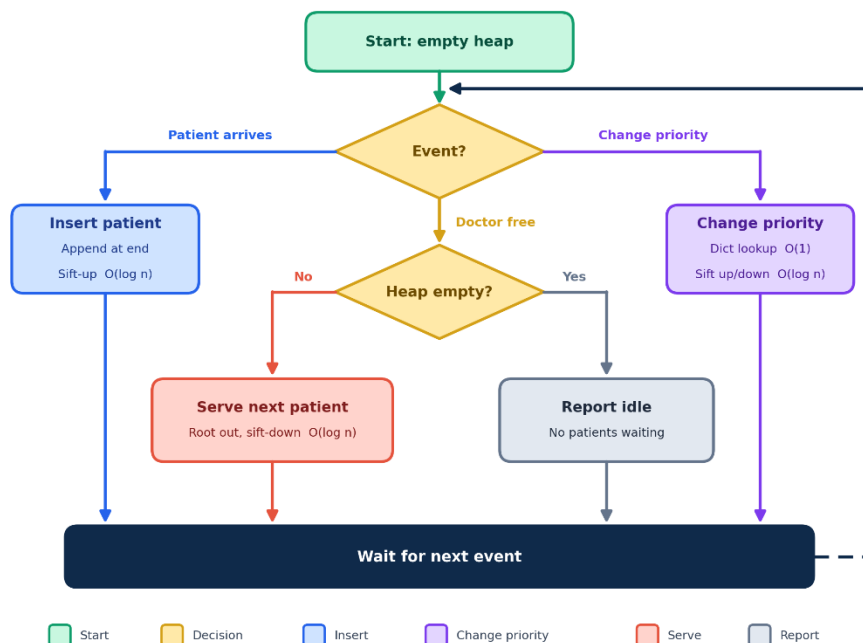
- Always find the next patient efficiently (no full scan or sort).
- Add, treat and change priority in $O(\log n)$ using a binary heap.
- Handle edge cases: empty queue, invalid priority, duplicate or unknown ID.

3. Proposed Approach

- Data structure: Binary heap used as a priority queue.
- Heap stored in a list: root (index 0) is always the next patient; $\text{parent} = (i-1)//2$, $\text{children} = 2i+1, 2i+2$.
- Rank = (-priority, arrival): higher priority comes first, earlier arrival breaks ties.
- A dictionary (where) stores each patient's heap index, so a patient is found instantly on priority change.
- Why heap: $O(1)$ to see the next patient, $O(\log n)$ to add or remove.

4. Algorithm / Pseudocode

1. Add: validate input, append patient at the end, record index in where, sift up.
2. Treat next: take the root, move the last patient to the root, sift down (None if empty).
3. Change priority: look up index in where, update value, sift up or down.
4. Repeat until the queue is empty.



5. Complexity Analysis

Complexity	Best Case	Average Case	Worst Case
Time	$O(1)$	$O(\log n)$	$O(\log n)$
Space	$O(n)$	$O(n)$	$O(n)$

- Add, treat_next and change_priority: $O(\log n)$, since the heap height is $\log n$.
- Peek: $O(1)$, the next patient is always at the root.
- Treating n patients: $O(n \log n)$ overall; space $O(n)$ for the heap and dictionary.

6. Source Code (Implementation)

- Python 3 (standard library only), VS Code.
- Classes: Patient (data + rank) and HospitalQueue (heap operations).

Heap and position map

```
py hospital_queue.py
35 s HospitalQueue:
36 def __init__(self):
37     self.heap = []           # the heap itself
38     self.where = {}         # patient ID -> index in self.heap
39     self.arrivals = 0       # counter that only goes up
```

Ranking rule

```
py hospital_queue.py
26 def rank(self):
27     # Smaller rank = treated earlier.
28     # -priority puts priority 5 before priority 1; arrival breaks ties.
29     return (-self.priority, self.arrival)
```

Sift up

```
py hospital_queue.py
49 while i > 0:
50     parent = (i - 1) // 2
51     if self.heap[i].rank() < self.heap[parent].rank():
52         self._swap(i, parent)
53         i = parent
54     else:
55         break
```

Sift down

```
py hospital_queue.py
59 n = len(self.heap)
60 while True:
61     left, right, best = 2 * i + 1, 2 * i + 2, i
62     if left < n and self.heap[left].rank() < self.heap[best].rank():
63         best = left
64     if right < n and self.heap[right].rank() < self.heap[best].rank():
65         best = right
66     if best == i:
67         break
68     self._swap(i, best)
69     i = best
```

Add

```
py hospital_queue.py
88 patient = Patient(pid, str(name).strip(), priority, self.arrivals)
89 self.heap.append(patient)
90 self.where[pid] = len(self.heap) - 1
91 self._sift_up(len(self.heap) - 1)
92 return patient
```

Treat next

```
py hospital_queue.py
100 if not self.heap:
101     return None
102 top = self.heap[0]
103 last = self.heap.pop()           # take the last patient out
104 del self.where[top.pid]
105 if self.heap:
106     self.heap[0] = last           # put him/her at the root
107     self.where[last.pid] = 0
108     self._sift_down(0)           # move down to the right place
109 return top
```

Change priority

```
py hospital_queue.py
115 if pid not in self.where:
116     return False
117 i = self.where[pid]
118 self.heap[i].priority = new_priority
119 self._sift_up(i)
120 self._sift_down(self.where[pid])
121 return True
```

7. Results and Test Cases

Sample run

```
TERMINAL
PS C:\hospital> python hospital_queue.py
Treat from empty queue -> None
Added P1 x (Priority 3) -> next: x
Added P2 y (Priority 5) -> next: y
Added P3 z (Priority 3) -> next: y
Added P4 p (Priority 1) -> next: y
Added P5 q (Priority 4) -> next: y
Added P6 r (Priority 2) -> next: y
Treatment order (x before z: same priority, arrived first):
1. P2 y (Priority 5)
2. P5 q (Priority 4)
3. P1 x (Priority 3)
4. P3 z (Priority 3)
5. P6 r (Priority 2)
6. P4 p (Priority 1)
p gets worse: priority 1 -> 5
1. P2 y (Priority 5)
2. P4 p (Priority 5)
3. P5 q (Priority 4)
4. P1 x (Priority 3)
5. P3 z (Priority 3)
6. P6 r (Priority 2)
Edge cases:
empty name: error - Name cannot be empty.
priority 9: error - Priority must be a whole number from 1 to 5.
duplicate ID: error - Patient ID P1 already exists.
unknown ID: -> False
Treating everyone:
1. Treated: P2 y (Priority 5) | still waiting: 5
2. Treated: P4 p (Priority 5) | still waiting: 4
3. Treated: P5 q (Priority 4) | still waiting: 3
4. Treated: P1 x (Priority 3) | still waiting: 2
5. Treated: P3 z (Priority 3) | still waiting: 1
6. Treated: P6 r (Priority 2) | still waiting: 0
```

- y (priority 5) is next; x is before z (same priority, arrived first).
- p moves from last to second after priority 1 -> 5, behind y (arrived earlier).
- Empty queue, empty name, priority 9, duplicate ID and unknown ID are handled safely.

8. Conclusion and Future Scope

- A min-heap on (-priority, arrival) always treats the most urgent, earliest patient first.
- Add, treat and change priority all run in $O(\log n)$; peek is $O(1)$.

9. References

- [1] T. H. Cormen et al., Introduction to Algorithms, 4th ed. MIT Press, 2022, ch. 6.
- [2] R. Sedgewick and K. Wayne, Algorithms, 4th ed. Addison-Wesley, 2011, sec. 2.4.